

1. Circuit Breaker

기존 스프링 AOP 방식의 서킷브레이커는 폴백 메소드 이름 오류, 낮은 응집도, 구현체 종속성 등의 문제가 있었습니다. 발표자는 이를 코틀린 함수와 체이닝 방식으로 개선하여 **코드 가독성과 유지보수성을 높이고, 서킷 오픈 시 별도 처리가 가능한 방식**을 제시했습니다.

1. 서킷 브레이커 사용 방식 개선하기

기존 스프링 AOP 방식의 서킷 브레이커는 여러 문제점을 가지고 있었지만, 코틀린 함수와 체이닝 방식을 활용하여 코드 가독성과 유지보수성을 높이고, 서킷 오픈 시 별도 처리가 가능한 방식으로 개선했다.

1.1. 서킷 브레이커란 무엇인가?

1. 서킷 브레이커의 역할

- 서버에서 특정 외부 호출이 문제를 일으켜 전체 서버 또는 API에 영향을 주는 것을 방지한다.
- 장애가 발생한 부분을 감지하여 더 이상 요청을 보내지 않도록 한다.

2. 작동 방식

- 실패 가능성이 있는 작업을 계속 요청하지 않기 위해 사용한다.
- 요청 대상이 정상 상태(클로즈 상태)일 때는 트래픽을 계속 흘려보낸다.
- 실패가 감지되면 서킷을 열어 더 이상 트래픽이 들어오지 않도록 차단한다.
- 이를 통해 장애로부터 회복할 시간을 벌고, 클라이언트에게는 빠른 실패를 전달한다.

1.2. 기존 서킷 브레이커 사용 방식

1. 스프링 클라우드 서킷 브레이커 인터페이스 활용

- 스프링에서는 서킷 브레이커 사용을 돕기 위해 스프링 클라우드 서킷 브레이커 인터페이스를 제공한다.
- 주로 리질리언스4j(Resilience4j)와 같은 라이브러리를 사용한다.
- **방법 1: 서킷 브레이커 직접 생성 및 사용**
 - 스프링 클라우드 서킷 브레이커 인터페이스의 팩토리로 서킷 브레이커를 직접 생성한다.

- 메인 로직을 서킷 브레이커로 감싸서 실행한다.
- 메인 로직에서 예외 발생 시 처리할 백오프(기본값 등)를 설정할 수 있다.
- **방법 2: 스프링 AOP 이용**
 - 서킷 브레이커를 적용하고 싶은 함수에 `@Breaker` 어노테이션을 사용한다.
 - 어노테이션에 서킷 브레이커 이름과 폴백 메소드 이름을 지정한다.
 - 메인 로직에서 에러 발생 시 지정된 폴백 메소드로 에러가 전달된다.
 - **폴백 메소드로 넘어오는 경우**
 - 메인 로직에서 예외가 발생한 경우: 발생한 예외를 그대로 전달받아 기본값 처리하거나 다시 던질 수 있다.
 - 서킷이 열린 경우: 메인 로직을 수행하지 않고 바로 폴백 메소드로 넘어오며, `CallNotPermittedException` 에러가 전달된다.
 - 서킷 오픈 시에는 서킷 오픈 시에만 동작하는 별도 로직을 추가할 수 있다.

1.3. 스프링 AOP 방식의 문제점

사용 방법

```

@CircuitBreaker(name = "name", fallbackMethod: "fallback")
fun findGroup(groupId: GroupId): Group {
    // main block
}

fun fallback(groupId: GroupId, e: Exception): Group {
    when (e) {
        is CallNotPermittedException -> return DefaultGroup()
        else -> throw e
    }
}

```

당근 SERVER MEETUP 2회



1. 컴파일 시 오류 감지 불가

- 폴백 메소드 이름이 문자열 형태로 들어가, 오타가 있어도 컴파일 단계에서 알 수 없고 런타임 예외가 발생할 수 있다.

2. 낮은 응집도

- 메인 로직과 폴백 함수 간의 관계를 파악하기 어렵고, 여러 폴백 중 어떤 것이 동작하는지 이름을 보고 찾아야 한다.

3. 복잡한 구성 및 구현체 종속성

- 서킷 브레이커를 별도로 구성하거나 특정 조건 로직을 넣는 것이 번거롭고 복잡하다.
- 함수 입장에서 리질리언스4j와 같은 특정 구현체를 직접 알아야 하는 문제가 있다.
- 서킷 오픈 시 발생하는 `CallNotPermittedException` 은 리질리언스4j에서 만든 예외이다.

4. 서킷 오픈 여부 구분 어려움

- 메인 로직에서 에러가 발생하면 항상 폴백으로 넘어오기 때문에, 서킷이 오픈되어 넘어온 것인지 일반적인 예외 상황인지 구분하는 과정이 필요하다.

5. 내부 함수 호출 제약

- 동일한 클래스의 내부 함수를 직접 호출하여 서킷 브레이커를 이용할 수 없다는 스프링 AOP의 기본적인 제약이 있다.

1.4. 개선된 서킷 브레이커 구현 방식

5. 내부 함수 호출 불가

```

fun findGroup(groupId: GroupId): Group {
    // ...
    val host = findUser(group.hostId)
    // ...
}

// circuit breaker not working
@CircuitBreaker(name = "name")
internal fun findUser(userId: UserId): User {
    val host = userFinder.find(userId)
    return host
}
                
```

당근 SERVER MEETUP 2회



1. 개선 목표 및 최종 모습 구상

- 서킷 브레이커 동작에 필요한 요소(메인 로직 감싸기, 서킷 브레이커 이름, 백엔드)를 고려했다.
- 최종적으로는 어노테이션 대신 코틀린 함수를 사용하여 선언적인 느낌을 주고 싶었다.
- 서킷 브레이커 이름은 함수에 전달하고, 백엔드 호출은 메인 로직 뒤에 체이닝되도록 구현하고자 했다.
- 서킷 브레이커가 오픈되었을 때만 동작하는 백엔드 함수를 추가하고 싶었다.
- API 패스가 자동으로 서킷 브레이커 이름으로 들어가도록 하고 싶었다.
- 함수 내부에서 여러 개의 서킷을 열어 사용할 수 있도록 구성하고 싶었다.

2. 핵심 구현 요소

• 서킷 함수 구현

- 스프링 클라우드 서킷 브레이커를 감싸기 위한 인터페이스와 구현체를 만들었다.
- 구현체는 스프링 클라우드 서킷 브레이커 팩토리를 받아 서킷 브레이커를 생성하고, 메인 로직을 감싸 실행한다.
- 서킷 브레이커 실행 시 `CallNotPermittedException` 이 발생하면, 이를 서킷 오픈 예외로 변환하여 내보낸다.
- `circuit` 함수는 메인 로직을 서킷 브레이커로 감싸 실행하며, API 패스를 기본 이름으로 사용하거나 직접 전달받을 수 있다.
- `DefaultCircuitBreakerProvider` 를 통해 싱글톤 인스턴스의 서킷 브레이커를 전달받아 구성한다.

• `Result` 와 `Try` 를 활용한 폴백 함수 구현

- 폴백 함수를 `circuit` 함수 뒤에 체이닝하기 위해, `circuit` 함수의 반환 타입을 `Result` 로 감쌌다.
- `Result` 는 성공 또는 실패(예외)를 담을 수 있는 코틀린 내장 클래스이다.
- `circuit` 함수의 응답을 `Try` 로 감싸 반환하며, 예외 발생 시 `Try` 의 실패로 감싸 응답한다.
- `fallback` 함수는 `Try` 의 확장 함수로 구현되어, `Try` 값이 실패인 경우에만 전달받은 백업 로직을 실행하고 `Try` 로 감싸 응답한다.
- `fallbackOverride` 함수는 `Try` 에서 꺼낸 예외가 서킷 오픈 예외일 경우에만 백업 로직을 실행한다.

- `runCatching` 활용

- 코틀린 전역 함수 `runCatching` 을 사용하면 직접 만든 `Try` 캐치와 동일하게 동작한다.

1.5. 개선된 방식의 장점

3. fallbackIfOpen

```
fun <T> Result<T>.fallbackIfOpen(block: () -> T): Result<T> {  
    when (this.exceptionOrNull()) {  
        is CircuitOpenException -> return runCatching { block() }  
        else -> return this  
    }  
}
```

당근 SERVER MEETUP 2회



1. 코드 가독성 및 유지보수성 향상

- 백엔드 함수의 이름을 잘못 적는 실수를 줄일 수 있다.
- 메인 로직과 폴백 함수 간의 응집도가 높아져 코드 파악이 용이하다.

2. 서킷 오픈 시 별도 처리 가능

- 서킷이 오픈되었을 때만 동작하는 백엔드 로직을 적용할 수 있다.

3. 선언적이고 간결한 코드

- 기존 스프링 AOP 방식은 `@Breaker` 어노테이션과 별도의 폴백 함수가 필요했지만, 개선된 방식은 함수 체이닝으로 간결하게 표현된다.
- 서킷 오픈 시 발생하는 특정 예외를 미리 알 필요가 없다.

1.6. TMI: Value 클래스와 스프링 연동 문제

완성

1. Fallback 이름 실수 → Fallback 이름 지정하지 않도록 변경
2. 낮은 응집도 → Fallback을 체이닝 함수로 변경
3. Open Fallback 처리 번거로움 → fallbackIfOpen 함수 구현
4. 구현체를 알아야 함 → Custom Exception으로 변환
5. 내부 함수 호출 불가 → 내부 함수 호출 가능

당근 SERVER MEETUP 2회



1. Value 클래스의 등장 배경

- API 응답 값을 클라이언트로 보내기 전에 조작하기 위해 Reified 라는 클래스를 만들었다.
- 이 클래스는 API 응답이 나가는 순간 응답 본문을 캐치하여 조작할 수 있도록 돕는다.

2. Value 클래스와 스프링 연동 시 문제 발생

- Value 클래스로 감싸진 응답을 기대했지만, 실제로는 Value 클래스가 사라지고 내부 값만 남아있었다.
- Value 클래스는 컴파일 과정에서 사라지고 프로퍼티만 남는 코틀린의 Value 클래스 특징 때문이다.
- 이는 스프링의 자바 코드와 코틀린 코드 간의 상호작용에서 발생하는 문제였다.

3. 문제 해결 방식

- 응답 바디를 Value 클래스로 한 번 더 감싸는 방식으로 해결했다.
- 이를 통해 Value 클래스의 get 함수를 통해 값에 접근하거나 예외를 던질 수 있게 되었다.
- 이 경험은 코틀린과 자바의 관계를 이해하는 데 도움이 되었다.